Writing a program always involves refining the abstractions as you go. When functions are the basic building block, this is done with the same basic function glue: invoking a function, passing a function as an argument, and returning a function.

By refining the abstractions in your program over time, you end up with a program that communicates what it does and that's easier to modify. Even better, it's the start of a library. To develop a really useful library, instead of starting with an idea for a library, extract it from a real project as an abstraction of ideas in that project. In listing 6.5 you can see the new version of the program.

When you run this new version of the program, you can view information about a specific product (such as the x1) by visiting the URL http://localhost:8080/product/x1. To see the profit for that same product at a given price point of 200, visit the URL http://localhost:8080/product/x1/profit?price=200.

The listing that follows is left *deliberately without any annotations*. The purpose of this is to show how far small named functions can go to make a program self-explanatory. Spend a bit more time to pore over this listing and learn how it works.

Listing 6.5. The improved program

```
http = require 'http'
url = require 'url'

{products, users} = require './db'

withCompleteBody = (req, callback) ->
  body = ''
  req.on 'data', (chunk) ->
    body += chunk.toString()
    request.on 'end', -> callback body

paramsAsObject = (params) ->
  pairs = params.split /&/g
  result = {}
  for pair in pairs
    splitPair = pair.split /\=/g
    result[splitPair[0]] = splitPair[1]
```